

When Private Blockchain Meets Deterministic Database

ZILIANG LAI, CHRIS LIU, and ERIC LO, The Chinese University of Hong Kong, China

Private blockchain as a replicated transactional system shares many commonalities with distributed database. However, the intimacy between private blockchain and deterministic database has never been studied. In essence, private blockchain and deterministic database both ensure replica consistency by determinism. In this paper, we present a comprehensive analysis to uncover the connections between private blockchain and deterministic database. While private blockchains have started to pursue deterministic transaction executions recently, deterministic databases have already studied deterministic concurrency control protocols for almost a decade. This motivates us to propose Harmony, a novel deterministic concurrency control protocol designed for blockchain use. We use Harmony to build a new relational blockchain, namely HarmonyBC, which features low abort rates, hotspot resiliency, and inter-block parallelism, all of which are especially important to disk-oriented blockchain. Empirical results on Smallbank, YCSB, and TPC-C show that HarmonyBC offers $2.0\times$ to $3.5\times$ throughput better than the state-of-the-art private blockchains.

CCS Concepts: • **Information systems** → **Database transaction processing**.

Additional Key Words and Phrases: Private Blockchain, Deterministic Database, Concurrency Control

ACM Reference Format:

Ziliang Lai, Chris Liu, and Eric Lo. 2023. When Private Blockchain Meets Deterministic Database. *Proc. ACM Manag. Data* 1, 1, Article 98 (May 2023), 28 pages. <https://doi.org/10.1145/3588952>

1 INTRODUCTION

A blockchain is a ledger *replicated* on every peer in the system. Internally, it has a *consensus layer* to ensure all non-faulty replicas agree on the same blocks of transactions and process each block of transactions with a *database layer*. The consensus layer is the bottleneck when the database layer is main-memory resident, i.e., data in DRAM and group commit logs to disk (e.g., ResilientDB [39]). However, the bottleneck stays in the database layer when the blockchain is disk-oriented [70]. Today, almost all enterprise-grade blockchains are disk-oriented (e.g., IBM Fabric [5], Facebook Diem [7], IBM RBC [57], ChainifyDB [69]) because of a lower deployment cost and various use cases (e.g., IoT device has limited memory [64], wimpy systems [52]). For disk-oriented blockchains, while prior work [57, 66, 70] has shown the database layer is the bottleneck (via overall throughput improvement when optimizing the database layer), we confirm that is still the case under a more recent setting with modern hardware, more nodes, and recent software version (Figure 1). Specifically, we show that recent (Byzantine) consensus protocols (e.g., HotStuff [81], GeoBFT [39], RCC [38]) have already outrun the disk-oriented database layers by an order of magnitude.

In this paper, we focus on disk-oriented blockchains (or simply *blockchains* hereafter) because of cost and adoption (e.g., Fabric is available on all major cloud providers [31] and has over 2000 companies deployed worldwide [45]). We aim to narrow the throughput gap between its database layer and the consensus layer. The consensus layer often collects transactions from clients and

Authors' address: Ziliang Lai, zllai@cse.cuhk.edu.hk; Chris Liu, cylui@cse.cuhk.edu.hk; Eric Lo, ericlo@cse.cuhk.edu.hk, The Chinese University of Hong Kong, Ho Sin-hang Engineering Building, Sha Tin, Hong Kong, China, 999077.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2836-6573/2023/5-ART98 \$15.00
<https://doi.org/10.1145/3588952>

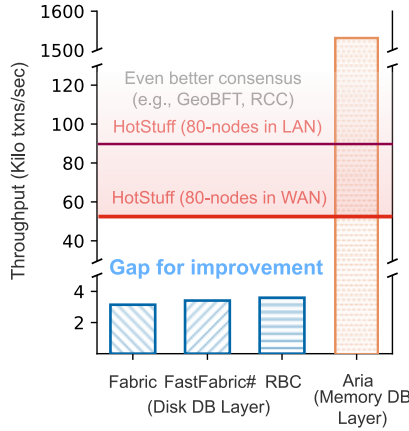


Fig. 1. The database layer is the bottleneck of disk-based private blockchains. Smallbank workloads on Amazon c5.4xlarge nodes. For WAN, 80 nodes are geo-distributed in four different continents. Throughputs of the database layers are measured by using only one ordering node to write off consensus.

broadcasts transaction blocks to all replicas [5, 57, 69]. Hence, as long as every replica receives and executes the same set of transactions in the database layer *deterministically*, all replicas will reach the same state, effectively achieving replica consistency. While concurrent transaction execution is the key to boosting the throughput of the database layer, classic concurrency control protocols are insufficient to uphold replica consistency due to non-determinism — given the same transaction block, replicas using the same concurrency control protocol may reach divergent states since they may get different serializable schedules. Consequently, we observe that state-of-the-art private blockchains are actually using different forms of *deterministic concurrency control* implicitly (perhaps subconsciously) to uphold replica consistency with concurrent execution.

The connection between private blockchain and *deterministic database* has never been explicit, although the connection between the former and *distributed database* has been studied elsewhere [65]. Private blockchain and deterministic database actually share a lot in common. First, both of them are distributed in nature. Second, they also process transactions block-by-block [27, 28, 53, 73]. Third, both blockchains and deterministic databases mandate consistent states across replicas with blockchains regarding replica consistency as a requirement while deterministic databases achieve replica consistency as a consequence.

The first contribution of this paper is to explicitly connect private blockchain with deterministic database. By exploiting the connections, we are able to fast-forward the development of private blockchain using lessons learned from deterministic databases. To demonstrate that, we follow some recent work [57, 69] to “chainify” (i.e., modifying a relational database to be a private blockchain [69]) PostgreSQL using the state-of-the-art deterministic concurrency control (DCC) protocol Aria [53]. Empirical results demonstrate that such a prototype already yields a $1.3\times$ – $2.7\times$ throughput improvement over the state-of-the-art private blockchains (Section 5). That is because Aria after all is the latest DCC protocol and it admits more concurrency than the ones used by the state-of-the-art private blockchains.

Although the preliminary results are exciting, we observe that existing deterministic concurrency control protocols are not optimized for a disk-oriented database layer. Hence, the second contribution of this paper is Harmony, a new deterministic concurrency control protocol designed for blockchains. Harmony features (i) low abort rates, (ii) hotspot resiliency, and (iii) inter-block

	Private blockchains				Deterministic databases		
	Fabric[5], Fabric++[70], Fabric#[66]	RBC[57]	ChainifyDB[69]		Calvin[73], BOHM[27], PWV[28]	Aria[53]	
Architecture	Simulate-Order-Validate		Order-Execute		Sequence-Execute		
Deterministic concurrency control	Optimistic		Pessimistic		Pessimistic		Optimistic
Storage	Disk-based				Memory-based		
Recovery	Physical logging		Logical logging		Logical logging		

Table 1. Private (disk-based) blockchains and Deterministic databases

parallelism, all of which are crucial to blockchains. First, compared to the state-of-the-art deterministic databases, Harmony is specially optimized to achieve low abort rates because an abort in a blockchain is way more expensive than an abort in a main-memory database — a transaction in blockchain involves not only expensive disk I/Os but also network round trips and cryptographic operations. Second, modern transactional workloads often suffer from hotspots [35]. A hotspot is a handful number of database records that are frequently updated. Hotspots could worsen the number of aborts of a concurrency control protocol. Deterministic concurrency control protocols, however, seldom optimize for hotspots (again due to aborts in main-memory database being relatively cheap) but Harmony’s goal of minimizing aborts can conveniently solve the problem. Third, unlike all deterministic databases, Harmony supports inter-block parallelism. Inter-block parallelism allows a block to start processing before its previous block ends. It is advantageous in blockchain because the complex interplay between I/O, network access, and caching would often result in high(er) transaction latency variance within a block. Hence, unless with inter-block parallelism, a blockchain would suffer from resource under-utilization that limits its throughput [57] (e.g., a straggler transaction in a block would idle CPU cores as well as block the pipeline). Overall, Harmony as a deterministic optimistic concurrency control protocol has overcome the challenge of reducing aborts with minimal serializability check overhead and supporting inter-block parallelism.

The last contribution of this paper is HarmonyBC, a private blockchain that chainifies PostgreSQL using Harmony. HarmonyBC inherits all the features from PostgreSQL such that it is one of the few blockchains that can support SQL and stored procedures as smart contracts. Most existing deterministic databases (e.g., Calvin [73], BOHM [27], PWV [28]) require a static analysis of the stored procedures to extract their read-write sets for deterministic scheduling. However, stored procedures often contain branches that predicate on the query results, which impede static analysis [53]. HarmonyBC does not have such a limitation. HarmonyBC is extensively evaluated using benchmarks commonly used in blockchains including YCSB [16] and SmallBank [58], and a relational benchmark – TPC-C [62]. Empirical results show that HarmonyBC offers 2.0× to 3.5× throughput better than RBC [57] and FastFabric# [66], and 2.3× throughput better than AriaBC (a blockchain implemented using the same framework as HarmonyBC but using Aria) under high contention.

The remainder of this paper is organized as follows. Section 2 analyzes the connections between private blockchain and deterministic database. Section 3 describes the design of our deterministic concurrency control protocol Harmony. In Section 4, we provide details of HarmonyBC. Section 5 presents the evaluation. Section 6 discusses related works and Section 7 concludes this paper.

2 CONNECTING PRIVATE BLOCKCHAIN WITH DETERMINISTIC DATABASE

Private blockchain can be used as a shared trustworthy ledger among a group of decentralized and untrusted companies while distributed databases cannot. Despite that, they share many commonalities as studied in [65]. In this paper, we point out an even closer relationship – private blockchain *is a* (secure) deterministic database in disguise. In this section, we present comparisons and analyze private (disk-based) blockchain and deterministic database. We focus on work that

optimizes the database layer in terms of architecture, concurrency, storage, and recovery. Table 1 gives a high-level summary. Discussions of other related work (e.g., consensus, sharding, use of SGX instructions) are in Section 6.

2.1 Architecture

There are two major types of architecture in private blockchains: Simulate-Order-Validate (SOV) [5, 66, 70] and Order-Execute (OE) [6, 57, 69]. Deterministic databases only have one type: Sequence-Execute (SE) [27, 28, 53, 73].

2.1.1 Simulate-Order-Validate (SOV). SOV is a blockchain architecture advocated by Fabric [5]. Fabric’s progeny such as Fabric++ [70] and Fabric# [66] all follow the same architecture. In SOV, a transaction has a workflow of “(1) client → (2) endorsers → (3) client → (4) orderer → (5) replicas” to ensure the transaction is consistently executed on all replicas.

First, (1) a client submits a transaction T to a subset of replicas (known as *endorsers*). (2) On arriving at an endorser, T begins its simulation phase and “executes” against the endorser’s local *latest state without persisting its writes*. The purpose of the simulation phase is to collect the read-write set of T , where the read-set contains keys and version numbers for all records read and the write-set contains the updated keys and their new values. Since individual endorsers may catch up with the latest states at different speeds, the read-write sets for the same transaction T may diverge across the replicas. Hence, SOV requires each endorser to sign the read-write set it produced and *sends back to the client*. After collecting the potentially diverged read-write sets of a transaction from the endorsers, (3) the client follows a predefined policy to pick one read-write set and sends it to an ordering service. (4) The ordering service is often an independent service that serves the consensus layer to collect transactions from all clients and form an agreed block of transactions. After that, the ordering service broadcasts transactions to all replicas in a block-wise manner. (5) On receiving a transaction block, a replica validates the security (e.g., verifying the signatures) and the serializability of the transactions in that block. For the latter, a replica checks if the received read-set of a transaction is still consistent with the replica’s local latest version (because it has been a while since the time of getting the read-set in Step (1)), aborts T if it has stale reads or commits T to update the replica’s local state otherwise.

2.1.2 Order-Execute (OE). OE is a blockchain architecture used in many recent private blockchains like RBC [57] and ChainifyDB [69]. In OE, clients submit transactions to an ordering service straight. The job of the ordering service in OE is the same as in SOV — collects client transactions, orders them to form transaction blocks, and broadcasts blocks to the replicas. Unlike SOV, OE only ships *transaction commands* around instead of shipping the transaction read-write sets, which saves network bandwidth. On receiving a block of transactions, each replica executes the block independently. To uphold consistency across replicas, one way is to enforce the individual replicas to honor the transaction order in the block by executing the transactions *serially* [6]. ChainifyDB and RBC, however, can execute transactions concurrently (Section 2.2).

2.1.3 Sequence-Execute (SE). SE is the architecture of all deterministic databases. In SE, there is a sequencing layer (which could be as lightweight as a single machine) collecting transactions from client applications and assigning a unique transaction ID (TID) to each transaction. Then, the sequencing layer broadcasts the transactions to the replicas in a block-wise manner. When a block of transactions reaches a replica, every replica executes the transactions using a *deterministic concurrency control* protocol, with the objective to reach the identical resulting states independently without any coordination.

Pessimistic DCC	Optimistic DCC
(a) Deterministic pre-scheduling - Lock manager (Calvin) - Multi-versioning (BOHM) - Dependency graph (PWV, ChainifyDB)	(c) Deterministic read-write sets - Extra-trip back to client (Fabric, Fabric++, Fabric#) - Snapshot-based (RBC, Aria)
(b) Deterministic schedule execution (Calvin, BOHM, PWV, ChainifyDB)	(d) Deterministic commit - Serial - Dangerous structure (Fabric, RBC) - Graph traversal (Fabric++, Fabric#) - Parallel - Dangerous structure (Aria)

Table 2. A taxonomy of DCC protocols

2.1.4 $OE = SE$. At this point, we can see that the order-execute architecture from blockchains is no different from the sequence-execute architecture from deterministic databases: in terms of functionality, the ordering service in OE is equivalent to the sequencing layer of deterministic databases, whereas the serial/concurrent transaction executions in private blockchains are actually different types of *deterministic concurrency control* in disguise.

2.2 Deterministic Concurrency Control (DCC)

Concurrency can improve system throughput if properly controlled. Deterministic databases have been focusing on *deterministic concurrency control* (DCC) to improve throughput, while the state-of-the-art private blockchains also have similar mechanisms (but not named DCC explicitly). By observing various sources of non-determinism in private blockchains and deterministic databases, we come up with a taxonomy unifying DCC protocols from both deterministic databases and private blockchains. Table 2 shows the taxonomy, in which a DCC protocol is either pessimistic or optimistic. Pessimistic DCC protocols carefully pre-define a concurrent schedule S for a block of transactions prior to their execution such that every replica can follow S to execute independently. Pre-defining a concurrent schedule for a block of transactions requires knowing all their potential conflicts a priori. Applying static analysis on a transaction block can achieve that to a certain extent. However, static analysis is insufficient when facing workloads with complex transaction logic (e.g., a stored procedure branches based on run-time query results), which is common in smart contracts. Optimistic DCC protocols provide deterministic concurrency at run-time, hence, it requires no static analysis and is thus more suitable to blockchains. Nonetheless, serializability conflicts may arise at run-time. Therefore, optimistic DCC protocols would require certain transactions to abort and restart to uphold serializability.

2.2.1 Pessimistic DCC (PDCC). The idea of pessimistic DCC is to pre-compute a concurrent schedule S for a block B of transactions. To achieve that, all pessimistic DCC protocols carry out static analysis on an incoming block B to obtain the read-write sets of its transactions to devise the schedule. Once the read-write sets of the transactions in the block are obtained, pessimistic DCC can devise a deterministic serializable schedule without any aborted transactions. In practice, however, the read-write sets of the transactions in the block are hard to get using static analysis on real smart contracts.

Deterministic pre-scheduling (Table 2a). Calvin [73] is an early deterministic database. Based on the read-write set from static analysis, it devises a concurrent schedule S for the lock manager to follow so that the lock manager can grant locks strictly based on the order imposed by the TIDs. BOHM [27] is another deterministic database. It “installs” the devised schedule S in its multi-versioned storage. It first creates a placeholder for every item that appears in the write-sets, with

its version number being the TID of the corresponding transaction. A transaction T_i (TID= i) then reads only items whose version number is the largest among all those smaller than i or waits until the desired placeholder is filled with a real value. PWV [28] is yet another deterministic database. It explicitly constructs a dependency graph based on the block's read-write sets and uses that to devise an explicit serializable schedule whose commit order follows the TIDs. ChainifyDB [69] is a recent private blockchain, which discusses how to turn an ordinary relational DBMS into a blockchain. Under the hood, ChainifyDB [69] is similar to PWV, which explicitly constructs dependency graphs and deterministic serializable schedules.

Deterministic schedule execution (Table 2b). Given a schedule S , as long as all replicas strictly honor S to execute, replica consistency can be achieved in a straightforward manner.

2.2.2 Optimistic DCC (ODCC). ODCC protocols provide deterministic concurrency at run-time and require no static analysis. Given a transaction T , ODCC first obtains its *deterministic read-write set*, despite that the states among the replicas may diverge due to different reasons (e.g., message delay). Since the deterministic read-write sets of transactions in a block may contain serializability conflicts, ODCC then follows a *deterministic commit* protocol to ensure serializability and replica consistency.

Deterministic read-write sets (Table 2c). SOV and OE (since OE=SE, from now on, we simply focus on OE) have different sources of non-determinism when obtaining the read-write sets.

For SOV-based private blockchains, since a transaction T is simulated on a replica using that replica's local *latest* state, T may get divergent read-write sets from different replicas because different replicas may catch up with the latest states at different rates. SOV pays the cost of an additional trip back to the client to reconcile the divergent read-write sets back to a deterministic one.

OE-based blockchains like RBC [57] and SE-based deterministic databases like Aria [53] use *block snapshots* to ensure all replicas get identical read-write sets. Specifically, they leverage the nature of block-based processing that the state after processing each block is identical across replicas. Hence, instead of obtaining the read-write set of a transaction based on the latest state of an individual replica, they obtain the read-write set of a transaction based on the state after executing a particular block. We call that state a *block snapshot*. A block snapshot is deterministic. Hence, individual replicas can always use a particular block snapshot as a single source of truth to execute transactions independently. For example, in Aria, a transaction T in block 10 would regard the snapshot of, say, block 9, as the block snapshot to execute on. Hence, despite T may get broadcasted to a replica R late (e.g., R has already received transactions from block 12) or early (e.g., R is still processing block 7), T would always get executed against the same snapshot from block 9. For the latter case, T would wait until the required snapshot is ready.

Deterministic commit (Table 2d). Given the identical read-write sets across all replicas, currently, all private blockchains conduct deterministic commit *serially*. Deterministic databases are way more advanced in this aspect. They are able to achieve *parallel commit*.

To ensure the commit process is deterministic, i.e., replicas independently can validate and abort/commit identical sets of transactions, all blockchains that adopt ODCC commit each transaction one by one, serially. In this regard, Fabric validates transactions in a block in the order of their TIDs and aborts a transaction on seeing a stale read. A stale read can be regarded as a "dangerous structure" [58] of read(r)-write(w)-dependency (i.e., a transaction reads a record that has been overwritten by another transaction). However, such a dangerous structure is often overly

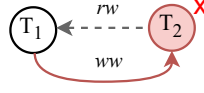


Fig. 2. Aria aborts a transaction on seeing a ww-dependency

conservative and leads to many false aborts. Specifically, consider two transactions T_1 and T_2 , where T_2 first reads x , followed by T_1 updates x but T_1 gets committed first. While the read of T_2 is a stale read (hence Fabric would abort T_2), T_2 indeed can still commit by viewing the resulting serializable schedule as $T_2 \rightarrow T_1$. Hence, RBC leverages that opportunity and validates transactions based on another dangerous structure derived from serializable snapshot isolation (SSI) [58]. RBC gets fewer false aborts. But it still needs to validate transactions serially to uphold determinism. Validation based on dangerous structures may lead to many false aborts because a dangerous structure can only capture local dependencies around a particular transaction, missing the big picture. Formally, serializability shall actually look for any cycle in the complete transaction dependency graph. Hence, private blockchains that enhance Fabric (e.g., Fabric++ [70] and Fabric# [66]) construct a dependency graph for the transactions in the ordering service and carry out early validation there. With a full picture of the dependency graph, there would be fewer aborts, and aborted transactions would not be included in a block and broadcast to the replicas¹. Nonetheless, the expensive and unparallelizable graph traversal would become a bottleneck when under high contention (Section 5).

Aria as the ODCC inside a deterministic database can commit transactions in parallel at the cost of more aborts. Specifically, one dangerous structure that Aria is based on is write(w)-write(w)-dependency (i.e., two transactions having overlapping write-sets). On seeing a ww-dependency, Aria aborts the one with a larger TID. For example, in Figure 2, Aria aborts T_2 on seeing $T_1 \xrightarrow{ww} T_2$. Breaking ww-dependencies not only avoids cycles (hence serializable) but also allows parallel commit — after validation, transactions that can be committed always have non-overlapping write sets and thus they can commit in parallel. However, it has a high abort rate in return — whenever multiple transactions are updating the same record, only one of them can commit. In fact, if the rw-dependency in Figure 2 does not actually exist, Aria still aborts T_2 even though there is no cycle.

2.3 Storage

Main-memory blockchains store data in memory and group commit logs to disk [39]. But enterprise-grade private blockchains are closer to traditional databases that store both data and log on disk and use DRAM for caching. Disk-oriented blockchains are prevalent in the market because of cost (Amazon sells DRAM DDR4 ECC for around USD 9.57/GB vs. NVMe SSD around USD 0.23/GB; 42× cost gap) and applications. Deterministic databases are closer to main-memory databases/blockchains that store data in main memory and use disk for logging. This explains why Aria as an ODCC protocol is not the best for our setting because transactions in main-memory databases are usually short-lived (in the order of μs) and thus weigh parallelism higher than aborts in their protocols [74].

2.4 Recovery

There are two classes of logging techniques for recovery: logical logging and physical logging. Logical logging logs only the transaction commands. Physical logging logs either the read-write

¹Hence, in Fabric++ and Fabric#, their validation phase only validates the security elements (e.g., signatures) but not serializability.

sets or ARIES-like redo-undo log. All SOV-based blockchains use physical logging. RBC as an OE-based blockchain also uses physical logging.

Deterministic databases use logical logging which has almost no runtime overhead. In distributed databases, parallel recovery from logical logs would generally incur non-determinism, because a replica may get recovered to another serializable database state unless specific treatments are provided [61]. But that would not happen in deterministic databases since it can concurrently replay the input transactions with determinism.

3 HARMONY

With the understanding of the close connections, private blockchains can actually be improved by lessons learned from deterministic databases. In this section, we present Harmony, a new deterministic concurrency control protocol that supports *parallel commit* with *no application limitation*. It is superior to the latest DCC Aria when applied to disk-oriented blockchain by featuring (i) *low abort rate* (ii) *hotspot resiliency* and (iii) *inter-block parallelism*. Note that improving the concurrency control of a disk-based database at first glance may seem to be a minor issue because a database's performance seems to be dominated by its disk latency. Yet, decades of concurrency control research has shown the otherwise because disk-based databases would use all sorts of techniques (e.g., DRAM buffer pools and group commit) to hide I/O latency. With those, higher concurrency (and thus higher CPU utilization) can be translated as better overlapping and I/O hiding, and reducing aborts can be translated as maximizing the useful work done per I/O, all of which could lead to higher throughput [33]. These factors motivate the design of better (deterministic) concurrency control for the database layer of disk-based blockchains.

3.1 Overview

Harmony is an optimistic deterministic concurrency control (ODCC) protocol. On receiving a block of transactions, it executes transactions in two steps: a simulation step and a commit step, without using static analysis. Like most ODCCs, its simulation step obtains *deterministic read-write sets* (Table 2c) by simulating transactions against the same block snapshot. After all transactions of the current block finish simulation, it enters the commit step to carry out *deterministic commit* (Table 2d).

To reduce the abort rate, recent private blockchains have been devoting efforts to exploring broader dependency information (e.g., Fabric++ [70] and Fabric# [66] traverse the whole dependency graph). However, that is unparallelizable with high overhead especially when the graph is large (Section 5). In contrast, Harmony upholds parallelism by only requiring each transaction to examine dangerous structures without cross-thread coordination. To reduce the abort rate, Harmony carries out *abort-minimizing validation* (Section 3.2) that *commits all transactions* as long as their rw-dependencies do not exhibit a kind of new “*backward dangerous structure*”, while all the other dependencies (e.g., ww-dependencies) are managed by *update reordering* (Section 3.3) to ensure serializability without any abort. Harmony ensures that the whole process is parallelizable and requires no graph traversal.

Harmony achieves good resource utilization by supporting *inter-block parallelism* (Section 3.4) – a straggler transaction in block i cannot detain the next block $(i + 1)$. Supporting that requires dealing with a non-deterministic view of inter-block dependency due to network asynchrony. For example, if T_1 in block i depends on T_2 in block $(i + 1)$ (it is possible when these two blocks are concurrent), a replica may miss such inter-block dependency if block $(i + 1)$ is delayed, causing it sees different dependencies from the other replicas. Harmony designs an *inter-block abort policy* to ensure deterministic commit under network asynchrony.

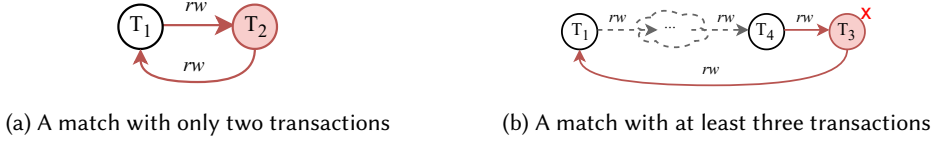


Fig. 3. Examples of backward dangerous structure

3.2 Abort-minimizing Validation

For ease of presentation, we assume inter-block parallelism is disabled in this section. In this case, a transaction T starts execution only after the previous block is finished and T 's simulation step reads the block snapshot of the previous block. We discuss the enabling of inter-block parallelism in Section 3.4.

Formally, there are three types of dependencies:

- *rw-dependency*: transaction T_j *rw-depends* on T_i if T_i reads any before-image of T_j 's writes, denoted as $T_i \xrightarrow{rw} T_j$ or equivalently $T_j \xleftarrow{rw} T_i$. The latter format is used when we discuss the "backward" dangerous structure later.
- *ww-dependency*: transaction T_j *ww-depends* on T_i if T_j overwrites any T_i 's write, denoted as $T_i \xrightarrow{ww} T_j$.
- *wr-dependency*: transaction T_j *wr-depends* on T_i if any of T_i 's writes is read by T_j , denoted as $T_i \xrightarrow{wr} T_j$.

We define *rw-subgraph* as the subgraph induced by *rw-dependency* edges in the dependency graph (i.e., the dependency graph with only *rw-dependencies*). Recall that to uphold serializability, no cycles can occur in the whole dependency graph. Harmony's abort-minimizing validation focuses on the *rw-subgraph* only because other dependencies would be handled by update reordering (Section 3.3) without any abort. It is based on the following rule:

RULE 1. (Validation Rule) A transaction T_j is aborted if it resides in a backward dangerous structure: $T_i \xleftarrow{rw} T_j \xleftarrow{rw} T_k$, $i < j$ and $i \leq k$.

Rule 1 is inspired by SSI (serializable snapshot isolation) [12, 30, 58], but with two notable differences: (1) it imposes a reverse order on the TIDs (where the oldest transaction T_k with the largest TID comes first and the youngest transaction T_i with the smallest TID comes last); and more importantly (2) it imposes no constraint on the *ww-dependencies* (i.e., non-first committer can still win). Figure 3 shows two examples that match the backward dangerous structure. Notice that a backward dangerous structure can be as small as having two transactions only since we allow $i = k$ (Figure 3a). A dangerous structure can also involve an arbitrary number of transactions (e.g., Figure 3b, where $T_1 \xleftarrow{rw} T_3 \xleftarrow{rw} T_4$). Specifically, given any cycle in the *rw-subgraph*, denote the transaction with the smallest TID in the cycle as T_i . Since T_i is in a cycle, there exists T_j such that $T_i \xleftarrow{rw} T_j$, and there also exists $T_j \xleftarrow{rw} T_k$. Thus, we have $T_i \xleftarrow{rw} T_j \xleftarrow{rw} T_k$, which is a backward dangerous structure because T_i has the smallest TID. Therefore, eliminating all the backward dangerous structures breaks all the cycles in the *rw-subgraph*.

Similar to Fabric [5], RBC [57], and Aria [53], Rule 1 could induce some false aborts (e.g., when the dashed arrows do not exist in Figure 3b). Eliminating all false aborts would require an expensive and unparallelizable graph traversal (e.g., Fabric++ [70] and FastFabric# [66]), which may outweigh the gain (Section 5). Nonetheless, Harmony can induce fewer false aborts than all the dangerous-structure-based ODCCs *in all cases*. Specifically, compared to Fabric who hastily aborts T_2 on seeing a *single* *rw-dependency* $T_1 \xleftarrow{rw} T_2$, Rule 1 is more judicious as it only aborts T_2 on seeing *both*

$T_1 \xleftarrow{rw} T_2$ and $T_2 \xleftarrow{rw} T_3$. Compared to RBC and Aria which also abort a transaction based on seeing a pair of rw-dependencies, Harmony would not abort on seeing a ww-dependency. For ww-dependencies, Harmony reorders the transactions to make them all able to commit, instead of aborting them (Section 3.3).

Algorithm 1: Harmony (no inter-block parallelism)

```

1 Initialize foreach transaction  $T_j$  do
2    $T_j.\text{min\_out} \leftarrow j + 1$ 
3    $T_j.\text{max\_in} \leftarrow -\text{inf}$ 
4 end
5  $\triangleright$  In the simulation step:
6 Event  $\text{on\_seeing\_rw\_dependency}(T_i \xleftarrow{rw} T_j)$ 
7    $T_j.\text{min\_out} \leftarrow \min(i, T_j.\text{min\_out})$ 
8    $T_i.\text{max\_in} \leftarrow \max(j, T_i.\text{max\_in})$ 
9 end
10  $\triangleright$  In the commit step:
11 Event  $\text{on\_entering\_commit}(T_j)$ 
12   if  $T_j.\text{min\_out} < j$  and  $T_j.\text{min\_out} \leq T_j.\text{max\_in}$  then
13      $\text{Abort}(T_j)$ 
14   else
15      $\text{Apply\_write\_sets}(T_j)$   $\triangleright$  See Section 3.3
16 end

```

Algorithm 1 shows the implementation of Rule 1. It contains two *event handlers* that would be invoked when a specific event happens (e.g., HarmonyBC is implemented using PostgreSQL's event API). In the simulation step, $\text{on_seeing_rw_dependency}()$ is invoked when a rw-dependency is found (line #6). The handler maintains two variables for each transaction:

- *minimal outgoing TID* of T_j : $\text{min_out} = \min\{i | T_i \xleftarrow{rw} T_j, i < j\}$. If no $T_i \xleftarrow{rw} T_j (i < j)$, define $\text{min_out} = (j + 1)$;
- *maximum incoming TID* of T_j : $\text{max_in} = \max\{k | T_j \xleftarrow{rw} T_k\}$. If no $T_j \xleftarrow{rw} T_k$, define $\text{max_in} = -\text{inf}$.

These two variables are used for checking Rule 1 in the commit step. When a transaction T_i enters the commit step, $\text{on_entering_commit}()$ is invoked and line #12 effectively checks Rule 1. If the transaction is not aborted, it invokes $\text{Apply_write_sets}()$ to apply the write-sets which is detailed in Section 3.3. Algorithm 1 is highly parallel because both the simulation step and the commit step can process transactions concurrently, and the event handlers could be triggered in parallel. We prove Algorithm 1 effectively checks Rule 1 as follows:

PROOF. We prove by showing that (a) a transaction aborted by Rule 1 is also aborted by Algorithm 1 and (b) vice versa. (a) Consider T_j is aborted due to a backward dangerous structure $T_i \xleftarrow{rw} T_j \xleftarrow{rw} T_k, i < j$ and $i \leq k$. By the definition of min_out , $\text{min_out} \leq i$; and by the definition of max_in , $\text{max_in} \geq k$. Thus, Algorithm 1 would abort T_j because it satisfies the condition in line #12. (b) Consider T_j is aborted by Algorithm 1 due to $\text{min_out} < j$ and $\text{min_out} \leq \text{max_in}$. There exists $T_{\text{min_out}}$ and $T_{\text{max_in}}$ such that $T_{\text{min_out}} \xleftarrow{rw} T_j \xleftarrow{rw} T_{\text{max_in}}$ is a backward dangerous structure. Thus T_j is also aborted by Rule 1. $\square$

Harmony does not have phantoms because $\text{on_seeing_rw_dependency}()$ would also be triggered by a predicate-read if it induces a rw-dependency. Let e be the number of rw-dependencies of

a transaction T_j . Algorithm 1 only takes $O(e)$ time because each rw-dependency of T_j is only examined once. In contrast, the validation phase in the original SSI takes $O(e^2)$ time [58] to check every pair of rw-dependencies incident to/from transaction T_j .

3.3 Update Reordering and Coalescence

Given an acyclic rw-subgraph, we now discuss how Harmony ensures the whole dependency graph is acyclic by *update reordering*, and how it achieves parallel commit using *update coalescence*. We still assume inter-block parallelism is disabled here.

3.3.1 Update Reordering.

Recall that Harmony's simulation step obtains deterministic read-write sets like other ODCCs. Harmony keeps the update commands (e.g., $\text{add}(x, 10)$) in the write-set instead of the updated values (e.g., $x = 20$). The update commands are collected in the simulation step when a transaction starts working on an UPDATE statement. For example, when T starts UPDATE bank SET balance = balance + 10 WHERE id = 'Alice', Harmony extracts the update command of $\text{add}(\text{Alice.balance}, 10)$ from the physical plan and stores it in T 's write-set without evaluating its value. Harmony ensures serializability by reordering the update commands using an efficient reordering algorithm. The update commands are then evaluated one after another following that order.

For example, assume T_1 and T_2 of the same block update x concurrently by $\text{add}(x, 10)$ and $\text{mul}(x, 3)$, respectively. Existing snapshot-based ODCCs (Table 2(c)) would evaluate both updates against the snapshot value of x (say $x = 10$). Thus, T_1 and T_2 would store $x = 20$ and $x = 30$ in their write-sets, respectively. However, one of T_1 and T_2 has to be aborted because neither $x = 20$ nor $x = 30$ is a serializable state of committing both (e.g., Aria aborts T_2 due to $T_1 \xrightarrow{ww} T_2$). In contrast, Harmony collects $\text{add}(x, 10)$ and $\text{mul}(x, 3)$ in the simulation step and uses the Reordering Rule (see below) to determine their order in the commit step. A correct order is necessary to ensure serializability because evaluating T_2 's update after T_1 induces two dependencies: (1) $T_1 \xrightarrow{ww} T_2$. Since T_2 's update command $\text{mul}(x, 3)$ is a read-modify-write operation, there is another dependency (2) $T_1 \xrightarrow{wr} T_2$. The induced dependencies could violate serializability if there is already a dependency $T_1 \xleftarrow{rw} T_2$ in the rw-subgraph, because it will form a cycle with (1) and/or (2). In this case, T_1 should update after T_2 such that $T_1 \xleftarrow{ww/rw} T_2$ is in the same direction as $T_1 \xleftarrow{rw} T_2$, which does not induce a cycle. Following this order both transactions can be committed: T_2 first updates $x = \text{mul}(x, 3) = 30$, and then T_1 evaluates $\text{add}(x, 10) = 40$. That is why Harmony does not have to abort a transaction on seeing a ww-dependency like Aria. As a tradeoff, evaluating the update commands one after another could impede parallelism. We use *update coalescence* to resolve that (Section 3.3.2).

Intuitively, the order of the update commands has to follow the direction of the dependencies in the rw-subgraph, such that the edges of ww and rw-dependencies also follow that direction to avoid cycles. The following theorem formalizes this principle.

THEOREM 1. *Given an acyclic rw-subgraph, the complete dependency graph is also acyclic if the update commands are reordered based on the topological order of the acyclic rw-subgraph.*

PROOF. (By contradiction) Assume there is still a cycle after reordering the update commands based on the topological order of the rw-subgraph. Then the cycle must contain $T_i \xrightarrow{wr/ww} T_j$ because the rw-subgraph is acyclic. We only consider T_i and T_j are in the same block because of no inter-block parallelism. (1) If $T_i \xrightarrow{ww} T_j$, T_j 's update is ordered after T_i . According to the definition of topological order, there is no directed path from T_j to T_i . Thus, $T_i \xrightarrow{ww} T_j$ is not in a cycle, which is

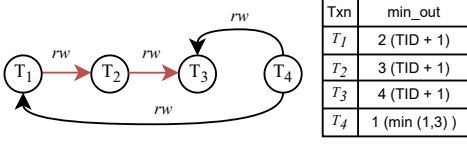


Fig. 4. An example of update reordering

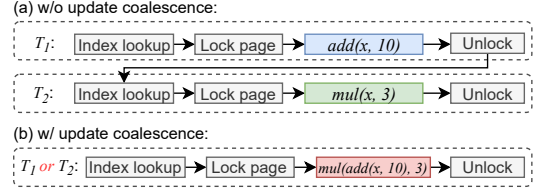


Fig. 5. Physical plans of the update commands with and without coalescence

a contradiction. (2) If $T_i \xrightarrow{wr} T_j$, such dependency is only possible when T_j has a read-modify-write update command and that command is ordered after T_i 's update. Therefore, T_j is ordered after T_i in the topological order. Similar to (1), $T_i \xrightarrow{wr} T_j$ is not in a cycle, which is also a contradiction. Hence, Theorem 1 is proved. $\square$

Although useful, the topological sort in Theorem 1 is expensive and hard to parallelize. Rule 1 now shines by not only ensuring an acyclic rw-subgraph, but also allowing the following Reordering Rule to avoid the topological sort.

RULE 2. (Reordering Rule) *Given all backward dangerous structures are eliminated after Rule 1, reorder the transactions that update the same record by the ascending order of their minimal outgoing TIDs (i.e., min_out), and break the tie by their own TIDs.*

Rule 2 transforms an expensive topological sort into a quick-sort (i.e., quick-sort by min_out), and it enables parallel reordering because updates on different records can be sorted in parallel. Figure 4 shows an example graph without backward dangerous structures. A topological sort on it results in $[T_4, T_1, T_2, T_3]$. Suppose only T_2 and T_4 update x , Harmony only needs to quick-sort the min_outs of T_2 and T_4 without traversing the whole graph like the topological sort. Nonetheless, the resulting order (i.e., $[T_4, T_2]$) is consistent with the topological ordering. Besides, if T_1 , T_2 , and T_3 update y , the two updater lists (i.e., $[T_2, T_4]$ on x and $[T_1, T_2, T_3]$ on y) can be sorted in parallel. It is easy to show that Rule 2 is deterministic because the ordering is based on TIDs that are consistent on all replicas. The following theorem shows the correctness of Rule 2.

THEOREM 2. *After applying Rule 1, the ascending order of min_outs is equivalent to a topological order of the rw-subgraph.*

PROOF. (By contradiction) Assume after applying the Validation Rule, $T_j.min_out < T_i.min_out$ but $T_j \xleftarrow{rw} T_i$ (i.e., min_out order is not consistent with the topological order of the rw-subgraph). (1) If there is no $T_k \xleftarrow{rw} T_j$ ($k < j$), by the definition of min_out, $T_j.min_out = j + 1$ and $T_i.min_out \leq j < T_j.min_out$. Therefore, it contradicts with $T_j.min_out < T_i.min_out$. (2) If there exists $T_k \xleftarrow{rw} T_j$ ($k < j$), by the definition of min_out, $T_j.min_out \leq k < j$ and $i > T_i.min_out$. By the definition of max_in, $T_j.max_in \geq i$. Therefore, $T_j.max_in \geq i > T_i.min_out > T_j.min_out$. Together with $j > T_j.min_out$, T_j should have been aborted by the Validation Rule. Thus it is a contradiction. $\square$

Reordering is a common technique to reduce aborts. Rather than reordering some dependencies after the values are computed [23, 53, 66, 70, 82], Harmony can reorder more by evaluating the commands in the commit step. IC3 [76] and ROCOCO [54] can achieve optimal reordering (i.e., minimal aborts). Yet they first need a static analysis to get the complete picture, which is not suitable for many blockchain workloads.

3.3.2 Update coalescence.

Although Rule 2 efficiently computes a serializable update order, efficiently enforcing that order when applying the update commands needs to solve two problems. First, transactions updating the same record would have unnecessarily duplicated disk I/Os and locking (problem P1). Figure 5 shows an example where T_1 and T_2 update x by $add(x, 10)$ and $mul(x, 3)$, respectively, and T_2 's update is ordered after T_1 . Without update coalescence (Figure 5a), T_2 's physical plan of updating x largely duplicates what T_1 does (e.g., index lookup and locking). Second, applying update commands one after another impedes parallelism (problem P2). In Figure 5a, since T_2 is ordered after T_1 , T_2 would be detained by T_1 especially when T_1 experiences disk I/Os or T_1 itself is detained by another transaction.

We propose update coalescence that merges multiple update commands on the same record into one to mitigate the above problems. Concretely, Harmony eliminates duplicated operations and merges the physical plans of updating the same record without affecting the semantics. As shown in Figure 5b, the physical plan after coalescing encodes both T_1 and T_2 's updates on x and respects the order of T_1 followed by T_2 . In this way, Harmony resolves P1. To resolve P2, Harmony dedicates only one transaction to apply the coalesced update for each record. In Figure 5b, if T_1 works on x first, it applies the coalesced update while T_2 can simply skip and work on other updates in parallel.

Algorithm 2: Update Reordering and Coalescence

```

1  ▶ In the simulation step:
2  Event on_update(key, update_command)
3      update_cmds ← update_reservation.search(key)
4      update_cmds.append(update_command)
5       $T_{current}.updated\_keys.append(key)$ 
6  end
7  ▶ In the commit step, invoked by Algorithm 1
8  Function Apply_write_sets( $T$ )
9      foreach key ∈  $T.updated\_keys$  do
10         update_cmds ← update_reservation.search(key)
11         if update_cmds.handled == False then
12             update_cmds.handled ← True
13             update_cmds.filter(update_command. $T.aborted$  == False)
14             update_cmds.sort_by(update_command. $T.min\_out$ )
15             coalesced_update ← coalesce(update_cmds)
16             apply(coalesced_update)
17         end
18     end
19 end

```

Algorithm 2 shows how update reordering and update coalescence fit into Harmony. In Algorithm 2, Harmony uses a hash table (*update_reservation*) to map a key to the list (*update_cmds*) of the update commands related to it. When the event handler *on_update*() is triggered (line #2), the update command is appended to the corresponding *update_cmds* list (line #4). The updated key is also appended to the *updated_keys* list of the transaction (line #5). The function *Apply_write_sets*(T) does the actual work of update reordering and coalescence, and it is invoked by Algorithm 1 after T passes the validation. For each updated key of T (line #9), the corresponding *update_cmds* is checked to see if it has already been handled by another transaction (line #11). Lines #11 and #12 ensure that for each updated record, only one transaction handles all updates related to it, and other transactions can skip and work on other updates in parallel (lines #11 and #12 are protected in a

critical section). To apply the updates, T first filters out the update commands added by the aborted transactions in the `update_cmds` (line #13), and then it performs update reordering by sorting the `update_cmds` based on the `min_outs` as suggested by Rule 2 (line #14). After that, it coalesces the update commands (as demonstrated in Figure 5) and applies the coalesced update (line #16).

There are two corner cases that are not explicitly handled in Algorithm 2: (1) T reads an x that has been updated by itself via a command $op(x)$; and (2) T updates x more than once. For (1), T retrieves $op(x)$ in the `update_reservation` table and evaluates it for the read operation. Notice that $op(x)$ may be evaluated twice in this case (i.e., once for the read in the simulation step and once after update reordering in the commit step), but both evaluations are guaranteed to return the same value since Rule 2 respects serializability. For (2), when T inserts the second update command to x 's `update_cmds` list, it coalesces the second update command with the first one such that the `update_cmds` effectively only contains at most one update command for each transaction.

With update reordering and coalescence, Harmony achieves hotspot resiliency because the former allows all concurrent updaters to commit, and the latter coalesces many updates of a hotspot into one. IC3 [76] also merges commands to reduce overhead. Yet, it is based on static analysis. Overall, update reordering and coalescence handle updates at the command level rather than at the value level, giving them a complete picture to optimize. Nonetheless, those opportunities might be lost in case a smart contract developer separates the read-modify-write logic. For example, consider `UPDATE ... SET X=X+1 WHERE Y=10`, which can be reordered and coalesce with other update statements. However, the opportunity would be lost if the developer expresses the logic as three pieces in a stored procedure: (1) first read the value of X using a SQL `SELECT`, (2) increment X , and (3) finally write the updated value of X using a SQL `UPDATE`. For some cases, the query optimizer may be able to rewrite them into one. Otherwise, smart contract developers are encouraged to express the entire read-modify-write logic as one SQL statement.

3.4 Inter-block Parallelism

Inter-block parallelism [5, 14, 57] aims to improve resource utilization in case a straggler transaction in block $(i - 1)$ detains the next block i . Concretely, when block $(i - 1)$ spares some resources (e.g., most transactions in block $(i - 1)$ are finished but a straggler is still running), Harmony would start some transactions from block i to utilize the spared resources before the whole block $(i - 1)$ finishes. When inter-block parallelism is enabled, transactions in block i carry out the simulation step based on the snapshot of block $(i - 2)$ instead of block $(i - 1)$, since the latter may not be ready when block i starts. Although adjacent blocks could run concurrently, Harmony still runs the commit step of block $(i - 1)$ before the commit step of block i to uphold determinism. As a tradeoff, inter-block parallelism may introduce inter-block dependencies that increase aborts. But our experiments (Section 5.7) confirm that better resource utilization outweighs that because Harmony is especially good at reducing the abort rate.

Harmony's validation (Rule 1) has to be enhanced to handle inter-block dependencies. For example, if T in block i reads x from the snapshot of block $(i - 2)$ and T' in block $(i - 1)$ updates x , there is an inter-block rw-dependency denoted as $T' \xleftarrow{\text{inter-rw}} T$ (we add a prefix of "intra" or "inter" for clarity). Moreover, inter-block dependencies may cause non-determinism due to network asynchrony. For example, Figure 6 shows a backward dangerous structure. A replica $R1$ who sees the whole backward dangerous structure would abort T_2 . However, if block $(i + 1)$ arrives late in replica $R2$ due to network asynchrony, $R2$ would commit T_2 because when applying Rule 1, $R2$ only sees $T_1 \xleftarrow{\text{intra-rw}} T_2$. This causes inconsistency between $R1$ and $R2$.

To deal with the inter-block dependencies (denoted as *inter-dep*), Harmony uses an enhanced validation rule.

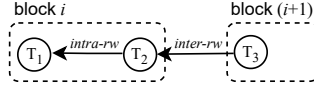


Fig. 6. An example that may cause non-determinism if without the Enhanced Validation Rule

RULE 3. (Enhanced Validation Rule) For each generalized backward dangerous structure:

$T_i \xleftarrow{\text{inter-dep/intra-rw}} T_j \xleftarrow{\text{inter-dep/intra-rw}} T_k, i < j \text{ and } i \leq k$, apply the following abort policy:

- (i) if T_j and T_k are in the same block, abort T_j ;
- (ii) otherwise, abort T_k .

Rule 3 is identical to Rule 1 if there are no inter-block dependencies. The additional abort policy (ii) ensures determinism despite network asynchrony. Specifically, consider the case in Figure 6, if block $(i + 1)$ is not delayed in the replica R1, it would abort T_3 instead of T_2 according to the abort policy (ii); suppose another replica R2 sees $T_1 \xleftarrow{\text{intra-rw}} T_2$ first and then sees $T_2 \xleftarrow{\text{inter-rw}} T_3$ later due to the delayed block $(i + 1)$. R2 would also abort T_3 . This ensures that R1 is consistent with R2. We now prove that Rules 3 and 2 together ensure serializability and they are deterministic.

PROOF. (1) Serializability: After applying both rules, we show that (a) there is no cycle within each block, and (b) there is no cycle across blocks. The correctness of (a) follows the fact that Rule 3 is equivalent to Rule 2 when no inter-block dependencies are considered. For (b), Rule 3 considers all the backward dangerous structures across blocks by adding inter-block dependencies into the generalized backward dangerous structure. Similar to how Rule 1 breaks the cycles in the rw-subgraph by eliminating backward dangerous structures, eliminating the generalized backward dangerous structure breaks the cycles across blocks. (2) Determinism: even with inter-block parallelism, Harmony enforces the blocks to enter their commit steps in the order of block ID. Thus, the blocks apply Rules 3 and 2 in a deterministic order. Since we have shown that Rules 3 and 2 are deterministic, we conclude that Harmony is deterministic with inter-block parallelism. $\square$

4 HARMONYBC

In this section, we describe in detail how we build a private blockchain, namely HarmonyBC, using Harmony DCC.

Architecture. Recall that private blockchains have two major architectures: Simulate-Order-Validate (SOV) and Order-Execute (OE). HarmonyBC adopts the OE architecture because it has no network overhead of sending large read-write sets. More importantly, Harmony DCC fits the OE architecture better because it only requires blocks of transaction commands as input.

Similar to the state-of-the-art relational private blockchains [57, 69], we implement Harmony DCC on PostgreSQL [55], a disk-based relational database as the database layer. Therefore, HarmonyBC has the full functionalities of a relational database. Since PostgreSQL is process-based, concurrency is achieved by executing all transactions in a block using multiple processes in parallel.

Consensus and fault-tolerance. The consensus layer of HarmonyBC is a pluggable module and we currently support the Byzantine-fault tolerant HotStuff [81] and crash-fault tolerant Kafka. Private blockchains typically rely on the consensus layer to uphold the safety and liveness guarantees [5, 37, 57]. With a BFT consensus layer (e.g., HotStuff [81]), HarmonyBC is also BFT because a faulty database node can only tamper its own state without affecting the non-faulty majority.

Recovery. HarmonyBC enjoys lightweight logical logging for crash-fault recovery because of determinism. It persists the small input blocks before execution instead of the large ARIES-like log. It utilizes the CHECKPOINT command in PostgreSQL to flush dirty pages to disk every p blocks

(e.g., $p = 10$). After checkpointing, the ID of the latest checkpointed block is also persisted into a `block_checkpoint_log`. During recovery, the replica loads the latest checkpoint and re-executes the blocks after the latest checkpointed block. When performing checkpointing, the previous checkpoint is not overwritten via PostgreSQL's multi-versioned storage. Therefore, if a replica crashes during checkpointing, it can still recover from the previous checkpoint.

Security. Private blockchains have to support node authentication to allow only identified nodes to join and must ensure tamper-proof [5]. For node authentication, we reuse the user authentication in the consensus layer (e.g., Kafka) such that only identified clients can submit transactions. The replicas are also authenticated when connecting to the consensus layer. For tamper-proof, since the input determines the final states in DCC, ensuring a tamper-proof input guarantees the tamper-proof of the final state. Therefore, HarmonyBC includes in each block a hash of the previous block like a typical blockchain, such that any tampered block could be identified by back-tracing the hash values from the latest block.

Limitations. Harmony inherits some limitations from the OE architecture. First, its trust model is fixed (i.e., majority) while SOV blockchains are able to program the trust logic in the *endorsement policy* (e.g., it can define one-vote veto). We plan to support it in future work by allowing users to define the endorsement policy via a stored procedure. Second, it does not support workload partitioning like SOV blockchains [5]. In future work, we also plan to employ sharding [3, 4, 21].

5 EVALUATION

We use two types of clusters. (1) Default cluster: 7 machines with Intel Xeon E5-2620v4, 64GB DRAM, and 800GB SSD; nodes are connected using 1Gbps Ethernet. This mimics the default setting of recent disk-based blockchains [57, 66, 70]. (2) Cloud cluster: 80 t3.2xlarge instances on AWS, each of which has 8 vCPUs, 32GB DRAM, and 30GB SSD. Nodes are either located in LAN (5Gbps Ethernet) or in WAN (across 4 regions on 4 different continents). We use the cloud cluster to study the scalability. All nodes run 64-bit CentOS 7.6 with Linux Kernel 3.10.0 and GCC 4.8.5.

We compare HarmonyBC with two SOV private blockchains: (1) Fabric v2.3 and (2) FastFabric# [66] (the latest progeny of Fabric; an optimized implementation of Fabric# [66]; also better than Fabric++ [70] and FastFabric [32]). We use the code provided by the author. We follow [13] and tune the system parameters to optimal. We also include an OE blockchain (3) RBC [57]. Since RBC is not open-sourced, we implemented it using the same framework as HarmonyBC. Besides, we also chainified PostgreSQL using Aria [53] as an OE private blockchain, namely (4) AriaBC. AriaBC is also implemented using the same framework as HarmonyBC because its original implementation is only an *in-memory* standalone implementation without integrating into a real system. Hence, all OE blockchains in this study are implemented using the same framework. All OE-based blockchain implementations (including HarmonyBC) provide the same security guarantees as the SOV-based blockchains and use the same consensus layer (Kafka by default). In fact, all SOV-based blockchains are given advantages because they use simpler key-value storage while RBC, AriaBC, and HarmonyBC are relational. Systems that require static analysis (Calvin [73], ChainifyDB [69], PWV [28], and etc [14, 27, 28, 59, 60, 77, 80]) are not included because of their application limitations. We exclude blockchains that rely on trusted hardware (e.g., CCF [68] uses SGX) because HarmonyBC does not make any assumption on trusted hardware.

We use YCSB [16], Smallbank [1], and TPC-C [62] benchmarks. For YCSB, we set the number of keys to 10K and follow [53, 75, 82] to wrap 10 operations into one transaction. Each operation has equal probabilities of being a simple SELECT or UPDATE. For Smallbank, we also set the number of accounts to 10K and use the standard mix. Unless stated otherwise, the block size is tuned to be optimal for each setting and the contention is set to medium (skewness = 0.6 in YCSB and Smallbank, warehouses = 20 in TPC-C). All workloads can exercise HarmonyBC's dangerous

skewness	0	0.2	0.4	0.6	0.8	1.0
YCSB	1.1%	1.2%	2.4%	9.9%	38.3%	74.3%
Smallbank	0.1%	0.1%	0.2%	1.5%	2.8%	10.6%
warehouses	1 20 40 60 80					
TPC-C	47.9% 8.8% 4.2% 2.7% 2.4%					

Table 3. Hit rate of the backward dangerous structure

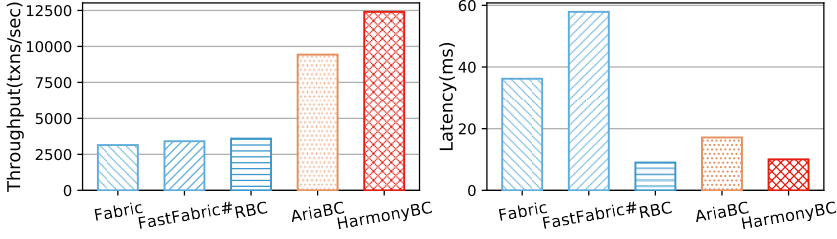


Fig. 7. Overall performance on Smallbank

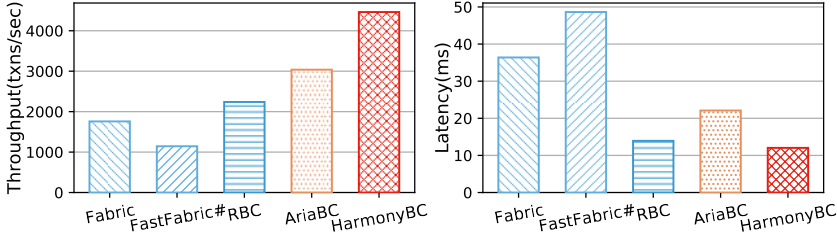


Fig. 8. Overall performance on YCSB

structure in various degrees (see Table 3). Since Fabric and FastFabric# are not relational (do not natively support TPC-C [48]), we use a separate section to discuss TPC-C results (Section 5.6), in which Fabric and FastFabric# are excluded.

5.1 Overall performance

In this experiment, we measure the peak throughput and the end-to-end latency of the committed transactions for all systems using the default cluster.

Figures 7 and 8 show the results. HarmonyBC attains 3.5× and 2.0× throughput over the best of the existing private blockchains (i.e., RBC in this experiment) in Smallbank and YCSB, respectively. It also achieves around 70% lower latency than SOV blockchains (Fabric and FastFabric#) because the OE architecture has fewer round-trips. RBC has similar latency as HarmonyBC but attains lower throughput because RBC admits a lower level of concurrency (see Section 5.2).

Simply implementing Aria as a private blockchain (i.e., AriaBC) readily yields better throughput than existing private blockchains, which shows the benefits of using an advanced DCC from a deterministic database to improve private blockchain. HarmonyBC attains a larger margin (e.g., 1.5× throughput over AriaBC in YCSB), demonstrating the effectiveness of HarmonyBC's blockchain-specific optimizations (e.g., low abort rate and inter-block parallelism). AriaBC has a slightly larger latency than HarmonyBC because its optimal block size is larger (see section 5.2)

We observe that FastFabric# outperforms Fabric in Smallbank but the opposite in YCSB. The reason is two-fold. First, the latest version of Fabric used in our experiments has a higher throughput than that used in FastFabric# paper. Second, a transaction accesses 10 records in YCSB (at most two

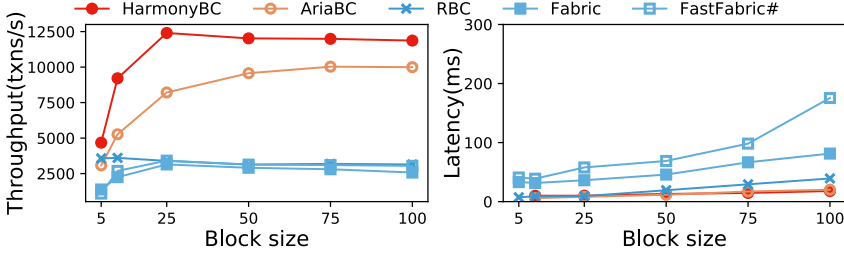


Fig. 9. Impact of block size on Smallbank

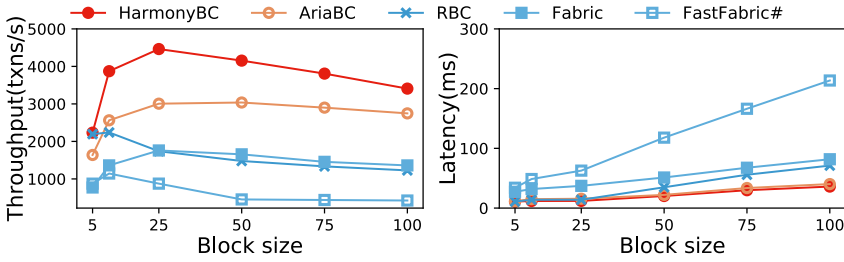


Fig. 10. Impact of block size on YCSB

in Smallbank), resulting in a much more complicated dependency graph. Our profiling shows the end-to-end throughput is bounded by the graph traversal indeed (a transaction spends around 75% of runtime on it). For the same reason, FastFabric# has higher latency than Fabric.

5.2 The impact of block size and concurrency

Figures 9 and 10 show the results when varying block sizes (number of transactions per block) in our default cluster. The block size is also the degree of concurrency for blockchains that support concurrent transactions (i.e., HarmonyBC, AriaBC, and RBC) because they can execute all transactions of a block in parallel using one process/thread per transaction.

When the block size is very small (e.g., 5), the throughput is low because of the limited degree of concurrency. The throughput also drops when the block size is too large, because of the increased number of conflicts and lock contentions. The throughput drops are less noticeable in Smallbank (an almost flat curve in Figure 9 when block size > 50) because transactions in Smallbank access fewer records and thus have fewer conflicts. The latency increases with the block size due to the longer time to form and process a larger block. This effect is most serious for FastFabric# because larger block sizes induce larger dependency graphs, causing it to spend more time on graph traversal.

HarmonyBC's optimal block size is 25 for both YCSB and Smallbank, while RBC has a smaller optimal block size (i.e., 10) because it commits transactions serially such that a larger block size does not increase the level of concurrency much. In contrast, AriaBC's optimal block size is larger (50 for YCSB and 75 for Smallbank) because it favors more concurrency since Aria is originally designed for a highly concurrent main-memory database.

5.3 The impact of contention and hotspot

We first study the impact of contention by varying the skewness of the accessed data in our default cluster. As shown in Figures 11 and 12, all systems incur more aborts with a larger skewness and thus their performances drop (less severe in Smallbank since it generally has lower contention

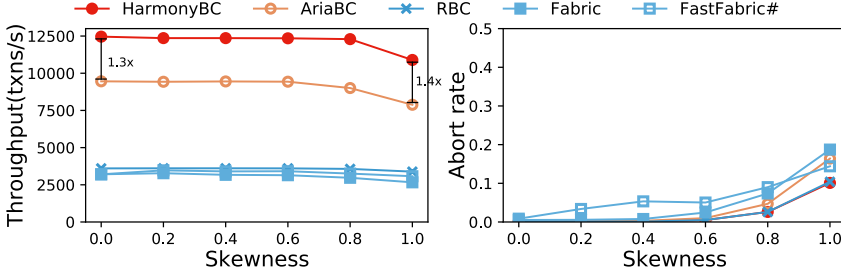


Fig. 11. Impact of contention on Smallbank

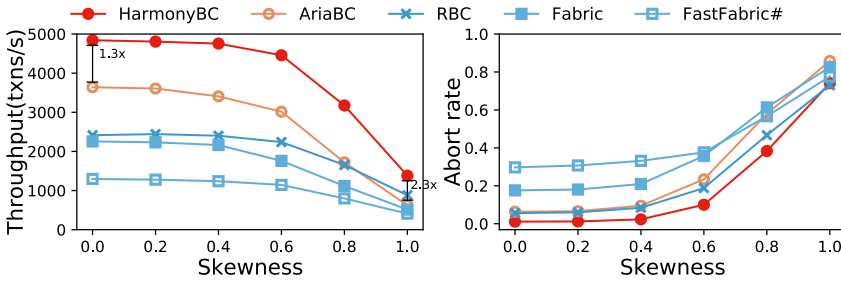


Fig. 12. Impact of contention on YCSB

compared to YCSB). Even when there is no skewness (i.e., skewness=0), Fabric and FastFabric# abort many transactions in YCSB because the non-deterministic read-write sets often fail the clients to find a legitimate one for committing the transaction. FastFabric# aborts more because in its implementation, it drops some transactions to avoid an overly large dependency graph. Since AriaBC and RBC abort a transaction on seeing a ww-dependency, and such dependencies occur fairly often even under low contention, HarmonyBC outperforms AriaBC and RBC under all skewnesses and it consistently has lower abort rates. Both AriaBC and HarmonyBC have low abort rates in Smallbank. But HarmonyBC outperforms AriaBC in throughput because it has fewer I/Os via update coalesce and better resource utilization via inter-block parallelism.

Figure 13 presents the false abort rates. In general, the false abort rate grows with higher contention except on YCSB with extreme contention (skewness=1.0) where most aborts in there are the real ones caused by serializability conflicts. Despite that, we observe that Harmony has the lowest false rates in all cases.

Next, we particularly study the hotspot resiliency of HarmonyBC by using a variant of YCSB (we exclude Smallbank because it is not contentious enough). There are still 10 statements in one transaction. But we set 1% of the records as hotspots, and a pair of SELECT and UPDATE operations would be rewritten as one UPDATE SQL statement that performs both read and write if they access the same record (since Postgres's optimizer does not have this rewrite rule, the hotspot resiliency feature of HarmonyBC has not been fully unleashed). Each statement accesses a hotspot with a controlled probability. Fabric and FastFabric# are not included in this experiment because they do not support SQL. As shown in Figure 14, HarmonyBC is almost unaffected by the hotspots, while the throughput of AriaBC and RBC drops significantly with increasingly higher abort rates when increasing the hotspot probability.

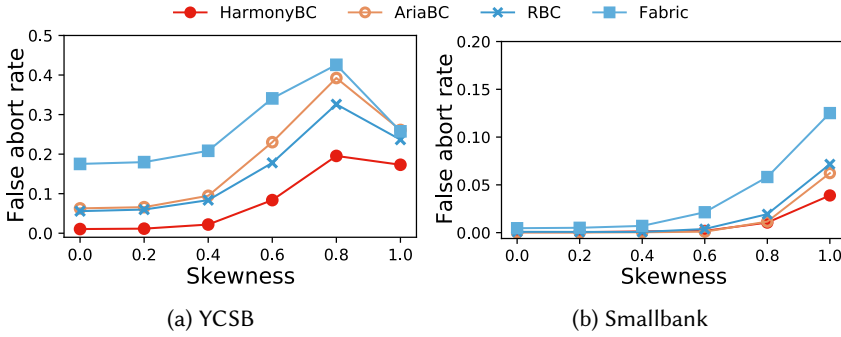


Fig. 13. False abort rate. FastFabric# is excluded because it uses an expensive unparallelizable graph traversal to eliminate all false aborts.

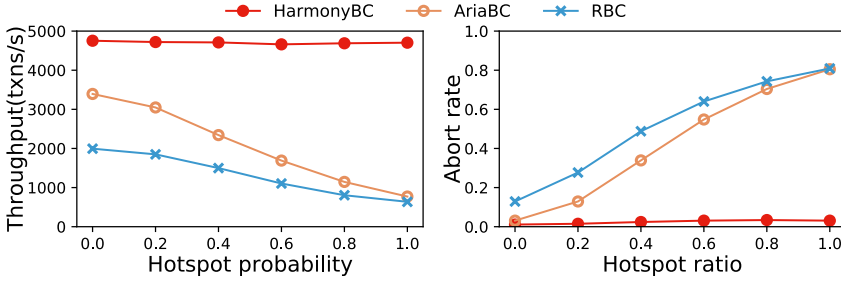


Fig. 14. Impact of hotspots

5.4 The impact of number of replicas

In this experiment, we scale the number of replicas up to 80 using the cloud cluster with 3 of them as the ordering service. All instances in this experiment are located in the same region. Figures 15 and 16 show the results. The throughput and latency of HarmonyBC, AriaBC, and RBC are almost unaffected by the number of replicas because their replicas work independently and only receive small transaction commands from the ordering service. All SOV blockchains, in contrast, exhibit throughput drop and latency increase with more replicas due to a larger network overhead of transferring read-write sets.

5.5 Geo-distributed cluster with BFT consensus

In this experiment, we study the performance of HarmonyBC using Byzantine-fault-tolerant (BFT) HotStuff [81] as the consensus layer and show that BFT would not hurt its throughput even scale up to 80 geo-distributed consensus nodes.

To do so, we use the cloud cluster and allocate 80 instances from 4 continents (Ohio, Mumbai, Sydney, and Stockholm; 20 instances each). For each instance, we run a consensus node and a replica node. Figures 17 and Figure 18 show the results. For comparison, the performance of the non-BFT HarmonyBC obtained in Section 5.4 is also shown in the figures. In the experiment, the first 20 instances are located in the same region (the white part), and experiments with more than 20 instances (the gray part) show the results of the geo-distributed setting. The results show that the throughput of HarmonyBC using a BFT consensus is almost unaffected (the minor throughput drop is due to HotStuff's other CPU overhead such as crypto). Since HotStuff requires more network round-trips to establish consensus than Kafka, the latency becomes larger and increases with more

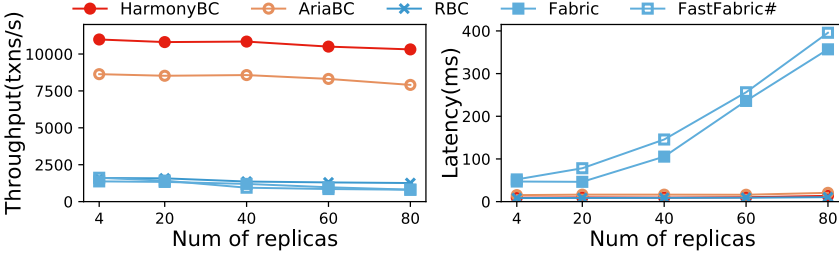


Fig. 15. Impact of number of replicas on Smallbank

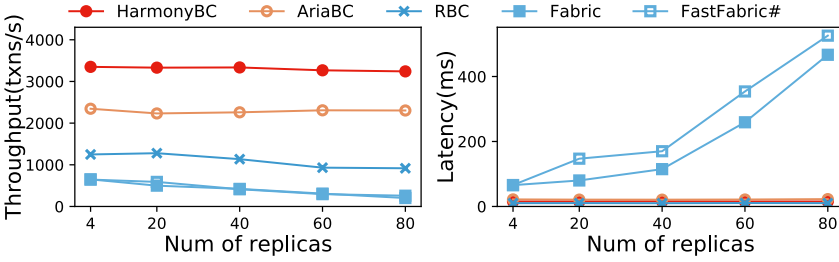


Fig. 16. Impact of number of replicas on YCSB

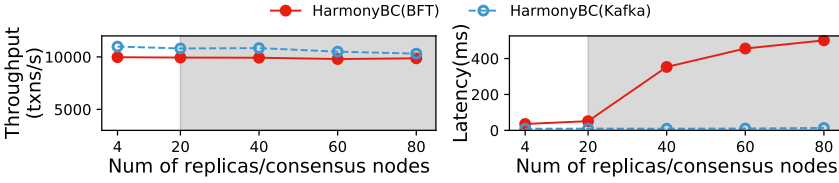


Fig. 17. Impact of BFT consensus on smallbank

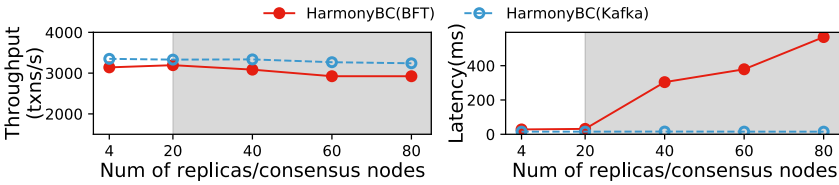


Fig. 18. Impact of BFT consensus on YCSB

nodes. However, the increased latency does not affect the throughput because the consensus layer is not the bottleneck in this setting.

5.6 TPC-C results

In this experiment, we study the performance of Harmony using TPC-C in our default cluster. Fabric and FastFabric# are excluded here because they do not support the relational model natively [48]. Figure 19 shows the results in TPC-C are consistent with the ones in YCSB and Smallbank, where HarmonyBC outperforms the others under all settings. HarmonyBC outperforms the others the most (3.3×) when the workload contention is the highest (1 warehouse), which reflects HarmonyBC's resiliency to hotspot and the effectiveness of its abort-minimizing design. Although with less contention, increasing the number of warehouses would increase the database size. That explains the throughput and latency trend for more than 20 warehouses.

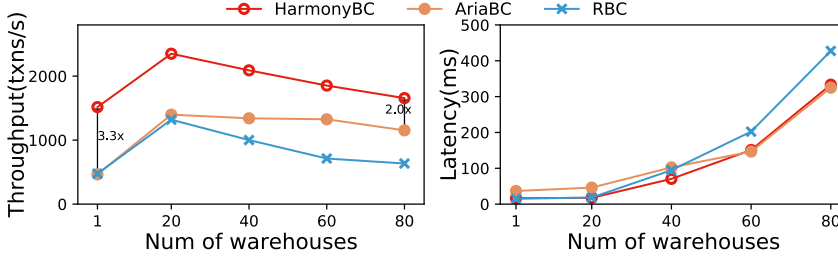


Fig. 19. TPC-C results

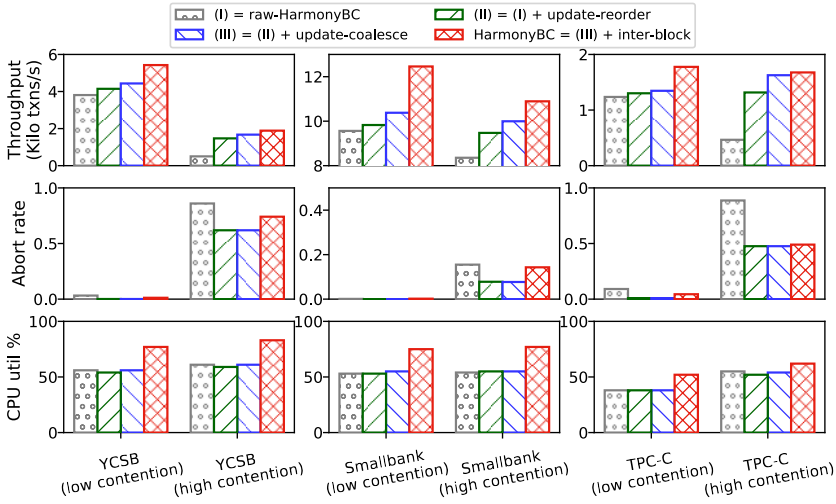


Fig. 20. Ablation Study under Low contention (0 skewness in YCSB/Smallbank; 80 warehouses in TPC-C) and High contention (1.0 skewness in YCSB/Smallbank; 1 warehouses in TPC-C)

5.7 Ablation study

In this experiment, we conduct an ablation study to demonstrate the effectiveness of each individual optimization technique. We cannot disable abort-minimizing validation because it is essential for correctness. When we disable update reordering, we use back Aria's style to abort a transaction on seeing a ww-dependency to preserve correctness.

Figure 20 shows the results, where raw-HarmonyBC means all optimizations of HarmonyBC are disabled (except abort-minimizing validation). From the top row, we see that each individual optimization can offer throughput improvement on all workloads, although to different degrees. Inter-block parallelism (red bar) and update-reordering (green bar) are shown to be the most effective optimizations under low contention and high contention, respectively. Inter-block parallelism improves low contention throughput by increasing CPU utilization (3rd row) whereas update-reordering improves high contention throughput by reducing the abort rate (2nd row). Notice that the use of inter-block parallelism increases the abort rate a bit, but it increases the overall throughput as a whole. It shows that better CPU utilization weighs higher than the other factors in this case.

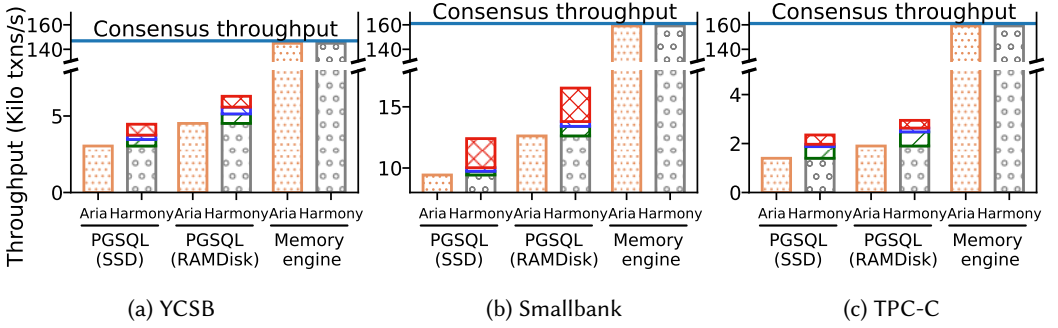


Fig. 21. Blockchain throughput difference between non-disk and disk-based database layer. The breakdown of Harmony's throughput is also given, following the format used in the ablation study. The consensus throughput varies across workloads due to the different transaction sizes.

5.8 Is Harmony still useful if all disk-related overheads are gone?

HarmonyBC is a disk-oriented blockchain. A natural question is whether Harmony's optimizations are still effective if all disk-related overheads in the database layer are gone. As pointed out by Stonebraker et al. [41], the cost spent on a disk-oriented database includes (i) the disk I/O latency and (ii) the cost of masking the I/O latency (e.g., buffer manager, 2PL). Hence, to answer that question, we carried out an experiment that replaces SSD with a RAMDisk to remove (i). PGSQL, the disk database that we build HarmonyBC on top of, has all sorts of costs on (ii) to minimize (i). Hence, we also implement Harmony's techniques as a standalone memory database engine to study the effect of Harmony without both (i) and (ii).

Figure 21 shows the results on all three workloads. For each workload, the two bars on the left are the results from the setting that we have been targeting – SSD-oriented blockchains. The two bars in the middle are the results of running the same but replacing the SSDs with RAMDisk. We can see that the throughput of HarmonyBC (as well as AriaBC) increases when there is no disk I/O. We can also see that all Harmony's optimizations are still effective there. With update reordering, fewer transactions are aborted; with inter-block parallelism, transactions from a later block can run with transactions from the previous block to increase CPU utilization. Both lead to higher throughput. Of course, as pointed out in [41], many components become unnecessary (e.g. buffer manager) or needed to be redesigned if using memory as the main storage. Those components exactly are the reasons why HarmonyBC's throughput cannot exceed the throughput of native memory blockchains (the two bars on the right). Yet, even though native memory blockchains can have the best throughput, their throughput is really capped by the consensus layer and it is still unclear whether the industry is willing to spend the DRAM money on that.

6 RELATED WORK

Blockchain consensus. Efforts have been devoted to improving the consensus layer of blockchains [8, 11, 17, 18, 36, 49, 63, 71, 84]. Most of them use an in-memory hash map to simulate the state machine [18, 36, 49, 63, 71]. Main-memory blockchains [38, 39] use DRAM as the storage layer and rely on replicas for fault tolerance – a setting orthogonal to Harmony. Without the disk-based database engine as the bottleneck, they can attain a throughput as high as 100K tps. Yet, disk-based blockchains still own the lion's share of the market given their economies of scale. HarmonyBC aims to narrow the performance gap between the database layer and the consensus layer.

Private Blockchain. Serial transaction execution cannot utilize multi-core CPUs for higher blockchain throughput (e.g., Quorum [6], Diem [7], Concord [34]). With a high-performance BFT

consensus layer, efficient deterministic concurrency becomes crucial to achieve better end-to-end throughput on disk-oriented private blockchains. Besides those covered in Section 2, Eve [46] is based on non-deterministic concurrency control. To upkeep replica consistency, it carries out consensus after concurrent execution. Since divergences happen frequently under non-deterministic execution, Eve also uses static analysis to avoid scheduling conflicting transactions to the same block, which inherits the application limitations from static analysis. SChain [14] is a private blockchain with inter-block parallelism, but it requires static analysis and hence with limited applications. Some private blockchains use sharding to scale out [2–4, 21, 26, 43, 72]. Sharding is part of our future work. Trusted hardware (e.g., Intel SGX) can also be utilized to improve blockchain performance [9, 20, 21, 51, 67] and privacy [10, 42, 68]. The use of trusted hardware is also orthogonal to Harmony.

Deterministic database. Other works in deterministic databases focus on contention management [60] and more efficient deterministic scheduling [59, 77, 80]. Some deterministic databases are able to eliminate two-phase commit (2PC) for cross-shard transactions [27, 28, 73]. However, they all require static analysis. Aria [53] does not require static analysis but then a light form of 2PC is required.

Distributed database. The connection between private blockchains and distributed databases has been studied both in terms of design [15, 19, 24, 65, 78, 79] and performance [25]. Fusing the optimization techniques from distributed database into private blockchains is an ongoing trend [32, 57, 66, 69, 70]. However, the close relationship between private blockchain and deterministic database has never been studied explicitly. We are the first to fill this gap and develop HarmonyBC to adopt deterministic concurrency control in private blockchains.

Database concurrency. Deterministic concurrency control is built on the shoulder of many modern concurrency control protocols [23, 35, 44, 47, 50, 56, 74, 75, 82, 83]. Harmony shares thoughts with serializable snapshot isolation (SSI) [12, 30, 58] in terms of simulating transactions against snapshots. However, SSI has a first-committer-wins rule that aborts a transaction if two transactions update the same record. In contrast, Harmony reorders updates on the same record without any abort. Harmony is also related to batch-based OCCs [23, 74] and group commit [22, 40]. They improve efficiency by batching some operations (e.g., logging) while transactions stream in. In contrast, Harmony’s input is already batched due to the blockchain setting. STOV2 [44] proposes commit-time update to reduce conflicts in OCCs. In [29], updates are even deferred until the updated records are read by the others. Harmony also applies updates during commit, but it is superior by further reducing conflicts using update reordering to re-organize the dependencies.

7 CONCLUSIONS

Many blockchains are still disk-oriented because of cost and use cases. In this paper, we connect private blockchain with deterministic database and propose Harmony, a deterministic concurrency control protocol that is inspired by the state-of-the-art deterministic databases but specifically optimized for blockchain’s disk-oriented database layer. HarmonyBC, our private blockchain prototype achieves 2.0× to 3.5× better throughput compared with the state-of-the-art private blockchains.

ACKNOWLEDGMENTS

We thank Prof. Beng Chin Ooi for inspiring us into this area. We also thank Dr. C. Mohan for his insightful comments on this work. This work is partially supported Hong Kong General Research Fund (14200817), Hong Kong AoE/P-404/18, Innovation and Technology Fund (ITS/310/18, ITP/047/19LP) and the Center for Perceptual and Interactive Intelligence (CPII) Ltd under InnoHK supported by the Innovation and Technology Commission (HKSAR Government).

REFERENCES

- [1] Mohammad Alomari, Michael Cahill, Alan Fekete, and Uwe Rohm. 2008. The cost of serializability on platforms that use snapshot isolation. In *2008 IEEE 24th International Conference on Data Engineering*. IEEE, 576–585.
- [2] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2019. CAPER: A Cross-Application Permissioned Blockchain. *Proc. VLDB Endow.* 12, 11 (2019), 1385–1398. <https://doi.org/10.14778/3342263.3342275>
- [3] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2019. On sharding permissioned blockchains. In *2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 282–285.
- [4] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2021. Sharper: Sharding permissioned blockchains over network clusters. In *Proceedings of the 2021 International Conference on Management of Data*. 76–88.
- [5] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. 2018. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the thirteenth EuroSys conference*. 1–15.
- [6] Arati Baliga, I Subhod, Pandurang Kamat, and Siddhartha Chatterjee. 2018. Performance evaluation of the quorum blockchain platform. *arXiv preprint arXiv:1809.03421* (2018).
- [7] Mathieu Baudet, Avery Ching, Andrey Chursin, George Danezis, François Garillot, Zekun Li, Dahlia Malkhi, Oded Naor, Dmitri Perelman, and Alberto Sonnino. 2019. State machine replication in the libra blockchain. *The Libra Assn., Tech. Rep* (2019).
- [8] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2015. Consensus-oriented parallelization: How to earn your first million. In *Proceedings of the 16th Annual Middleware Conference*. 173–184.
- [9] Marcus Brandenburger, Christian Cachin, Rüdiger Kapitza, and Alessandro Sorniotti. 2018. Blockchain and trusted computing: Problems, pitfalls, and a solution for hyperledger fabric. *arXiv preprint arXiv:1805.08541* (2018).
- [10] Marcus Brandenburger, Christian Cachin, Rüdiger Kapitza, and Alessandro Sorniotti. 2019. Trusted computing meets blockchain: Rollback attacks and a solution for hyperledger fabric. In *2019 38th Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 324–32409.
- [11] Yehonatan Buchnik and Roy Friedman. 2020. FireLedger: A High Throughput Blockchain Consensus Protocol. *Proc. VLDB Endow.* 13, 9 (2020), 1525–1539. <https://doi.org/10.14778/3397230.3397246>
- [12] Michael J Cahill, Uwe Röhm, and Alan D Fekete. 2009. Serializable isolation for snapshot databases. *ACM Transactions on Database Systems (TODS)* 34, 4 (2009), 1–42.
- [13] Jeeta Ann Chacko, Ruben Mayer, and Hans-Arno Jacobsen. 2021. Why do my blockchain transactions fail? A study of hyperledger fabric. In *Proceedings of the 2021 International Conference on Management of Data*. 221–234.
- [14] Zhihao Chen, Haizhen Zhuo, Quanqing Xu, Xiaodong Qi, Chengyu Zhu, Zhao Zhang, Cheqing Jin, Aoying Zhou, Ying Yan, and Hui Zhang. 2021. SChain: A Scalable Consortium Blockchain Exploiting Intra- and Inter-Block Concurrency. *Proc. VLDB Endow.* 14, 12 (2021), 2799–2802. <http://www.vldb.org/pvldb/vol14/p2799-chen.pdf>
- [15] Mohammad Javed Morshed Chowdhury, Alan Colman, Muhammad Ashad Kabir, Jun Han, and Paul Sarda. 2018. Blockchain versus database: A critical analysis. In *2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*. IEEE, 1348–1353.
- [16] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.
- [17] James A. Cowling, Daniel S. Myers, Barbara Liskov, Rodrigo Rodrigues, and Liuba Shrira. 2006. HQ Replication: A Hybrid Quorum Protocol for Byzantine Fault Tolerance. In *7th Symposium on Operating Systems Design and Implementation (OSDI '06), November 6-8, Seattle, WA, USA*, Brian N. Bershad and Jeffrey C. Mogul (Eds.). USENIX Association, 177–190. <http://www.usenix.org/events/osdi06/tech/cowling.html>
- [18] Tyler Crain, Christopher Natoli, and Vincent Gramoli. 2021. Red belly: A secure, fair and scalable open blockchain. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 466–483.
- [19] Michael Crosby, Pradan Pattanayak, Sanjeev Verma, Vignesh Kalyanaraman, et al. 2016. Blockchain technology: Beyond bitcoin. *Applied Innovation* 2, 6-10 (2016), 71.
- [20] Hung Dang, Anh Dinh, Ee-Chien Chang, and Beng Chin Ooi. 2018. Chain of trust: Can trusted hardware help scaling blockchains. *arXiv preprint arXiv:1804.00399* (2018).
- [21] Hung Dang, Tien Tuan Anh Dinh, Dumitrel Loghin, Ee-Chien Chang, Qian Lin, and Beng Chin Ooi. 2019. Towards scaling blockchain systems via sharding. In *Proceedings of the 2019 international conference on management of data*. 123–140.
- [22] Justin DeBrabant, Andrew Pavlo, Stephen Tu, Michael Stonebraker, and Stan Zdonik. 2013. Anti-caching: A new approach to database management system architecture. *Proceedings of the VLDB Endowment* 6, 14 (2013), 1942–1953.
- [23] Bailu Ding, Lucja Kot, and Johannes Gehrke. 2018. Improving optimistic concurrency control through transaction batching and operation reordering. *Proceedings of the VLDB Endowment* 12, 2 (2018), 169–182.

- [24] Tien Tuan Anh Dinh, Rui Liu, Meihui Zhang, Gang Chen, Beng Chin Ooi, and Ji Wang. 2018. Untangling blockchain: A data processing view of blockchain systems. *IEEE transactions on knowledge and data engineering* 30, 7 (2018), 1366–1385.
- [25] Tien Tuan Anh Dinh, Ji Wang, Gang Chen, Rui Liu, Beng Chin Ooi, and Kian-Lee Tan. 2017. Blockbench: A framework for analyzing private blockchains. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1085–1100.
- [26] Muhammad El-Hindi, Carsten Binnig, Arvind Arasu, Donald Kossmann, and Ravi Ramamurthy. 2019. BlockchainDB: A shared database on blockchains. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1597–1609.
- [27] Jose M Faleiro and Daniel J Abadi. 2015. Rethinking serializable multiversion concurrency control. *Proceedings of the VLDB Endowment* 8, 11 (2015).
- [28] Jose M Faleiro, Daniel J Abadi, and Joseph M Hellerstein. 2017. High performance transactions via early write visibility. *Proceedings of the VLDB Endowment* 10, 5 (2017).
- [29] Jose M. Faleiro, Alexander Thomson, and Daniel J. Abadi. 2014. Lazy Evaluation of Transactions in Database Systems. In *SIGMOD*. 15–26.
- [30] Alan Fekete, Dimitrios Liarokapis, Elizabeth O’Neil, Patrick O’Neil, and Dennis Shasha. 2005. Making snapshot isolation serializable. *ACM Transactions on Database Systems (TODS)* 30, 2 (2005), 492–528.
- [31] Hyperledger Foundation. 2019. Hyperledger Annual Report. <https://www.hyperledger.org/learn/publications/hyperledger-annual-report> (2019).
- [32] Christian Gorenflo, Stephen Lee, Lukasz Golab, and Srinivasan Keshav. 2019. Fastfabric: Scaling hyperledger fabric to 20,000 transactions per second. In *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 455–463.
- [33] James N Gray. 1978. Notes on data base operating systems. *Operating systems* (1978), 393–481.
- [34] GG Gueta, I Abraham, S Grossman, D Malkhi, B Pinkas, MK REITER, DA SEREDINSCHI, O Tamir, and A Tomescu. 1804. Sbft: a scalable decentralized trust infrastructure for blockchains, 2018.
- [35] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *Proceedings of the 2021 International Conference on Management of Data*. 658–670.
- [36] Suyash Gupta, Jelle Hellings, Sajjad Rahnama, and Mohammad Sadoghi. 2019. Proof-of-execution: Reaching consensus through fault-tolerant speculation. *arXiv preprint arXiv:1911.00838* (2019).
- [37] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. Fault-Tolerant Distributed Transactions on Blockchain. Morgan & Claypool.
- [38] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: Resilient Concurrent Consensus for High-Throughput Secure Transaction Processing. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 1392–1403. <https://doi.org/10.1109/ICDE51399.2021.00124>
- [39] Suyash Gupta, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2020. ResilientDB: Global Scale Resilient Blockchain Fabric. *Proc. VLDB Endow.* 13, 6 (2020), 868–883. <https://doi.org/10.14778/3380750.3380757>
- [40] Robert Hagmann. 1987. Reimplementing the Cedar file system using logging and group commit. In *Proceedings of the eleventh ACM Symposium on Operating systems principles*. 155–162.
- [41] Stavros Harizopoulos, Daniel J. Abadi, Samuel Madden, and Michael Stonebraker. 2019. OLTP through the looking glass, and what we found there. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, Michael L. Brodie (Ed.). ACM / Morgan & Claypool, 409–439. <https://doi.org/10.1145/3226595.3226635>
- [42] Mike Hearn and Richard Gendal Brown. 2016. Corda: A distributed ledger. *Corda Technical White Paper* 2016 (2016).
- [43] Jelle Hellings and Mohammad Sadoghi. 2021. Byshard: Sharding in a byzantine environment. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2230–2243.
- [44] Yihe Huang, William Qian, Eddie Kohler, Barbara Liskov, and Liuba Shrira. 2020. Opportunities for optimism in contended main-memory multicore transactions. *Proceedings of the VLDB Endowment* 13, 5 (2020), 629–642.
- [45] HG Insights. 2022. Companies Currently Using Hyperledger Fabric. <https://www.hyperledger.org/learn/publications/hyperledger-annual-report> (2022).
- [46] Manos Kapritsos, Yang Wang, Vivien Quema, Allen Clement, Lorenzo Alvisi, and Mike Dahlin. 2012. All about Eve:Execute-Verify Replication for Multi-Core Servers. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. 237–250.
- [47] Kangnyeon Kim, Tianzheng Wang, Ryan Johnson, and Ippokratis Pandis. 2016. Ermia: Fast memory-optimized database system for heterogeneous workloads. In *Proceedings of the 2016 International Conference on Management of Data*. 1675–1687.
- [48] Attila Klenik and Imre Kocsis. 2022. Porting a benchmark with a classic workload to blockchain: TPC-C on Hyperledger Fabric. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*. 290–298.

- [49] Ramakrishna Kotla, Lorenzo Alvisi, Michael Dahlin, Allen Clement, and Edmund L. Wong. 2007. Zyzzyva: speculative byzantine fault tolerance. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*, Thomas C. Bressoud and M. Frans Kaashoek (Eds.). ACM, 45–58. <https://doi.org/10.1145/1294261.1294267>
- [50] Hyeontaek Lim, Michael Kaminsky, and David G Andersen. 2017. Cicada: Dependably fast multi-core in-memory transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 21–35.
- [51] Jian Liu, Wenting Li, Ghassan O Karame, and N Asokan. 2018. Scalable byzantine consensus via hardware-assisted secret sharing. *IEEE Trans. Comput.* 68, 1 (2018), 139–151.
- [52] Dumitrel Loghin, Gang Chen, Tien Tuan Anh Dinh, Beng Chin Ooi, and Yong Meng Teo. 2019. Blockchain goes green? An analysis of blockchain on low-power nodes. *arXiv preprint arXiv:1905.06520* (2019).
- [53] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: a fast and practical deterministic OLTP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2047–2060.
- [54] Sujaya Maiyya, Faisal Nawab, Divy Agrawal, and Amr El Abbadi. 2019. Unifying Consensus and Atomic Commitment for Effective Cloud Data Management. *Proc. VLDB Endow.* 12, 5 (2019), 611–623. <https://doi.org/10.14778/3303753.3303765>
- [55] Bruce Momjian. 2001. *PostgreSQL: introduction and concepts*. Vol. 192. Addison-Wesley New York.
- [56] Shuai Mu, Yang Cui, Yang Zhang, Wyatt Lloyd, and Jinyang Li. 2014. Extracting more concurrency from distributed transactions. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 479–494.
- [57] Senthil Nathan, Chander Govindarajan, Adarsh Saraf, Manish Sethi, and Praveen Jayachandran. 2019. Blockchain meets database: design and implementation of a blockchain relational database. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1539–1552.
- [58] Dan RK Ports and Kevin Grittnr. 2012. Serializable Snapshot Isolation in PostgreSQL. *Proceedings of the VLDB Endowment* 5, 12 (2012).
- [59] Tamir M Qadah and Mohammad Sadoghi. 2018. Quecc: A queue-oriented, control-free concurrency architecture. In *Proceedings of the 19th International Middleware Conference*. 13–25.
- [60] Dai Qin, Angela Demke Brown, and Ashvin Goel. 2021. Caracal: Contention Management with Deterministic Concurrency Control. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 180–194.
- [61] Dai Qin, Ashvin Goel, and Angela Demke Brown. 2017. Scalable Replay-Based Replication For Fast Databases. *Proc. VLDB Endow.* 10, 13 (2017), 2025–2036. <https://doi.org/10.14778/3151106.3151107>
- [62] Francois Raab. 1993. TPC-C-The Standard Benchmark for Online transaction Processing (OLTP).
- [63] Sajjad Rahnama, Suyash Gupta, Rohan Sogani, Dhruv Krishnan, and Mohammad Sadoghi. 2021. RingBFT: Resilient Consensus over Sharded Ring Topology. *arXiv preprint arXiv:2107.13047* (2021).
- [64] Ana Reyna, Cristian Martín, Jaime Chen, Enrique Soler, and Manuel Díaz. 2018. On blockchain and its integration with IoT. Challenges and opportunities. *Future generation computer systems* 88 (2018), 173–190.
- [65] Pingcheng Ruan, Tien Tuan Anh Dinh, Dumitrel Loghin, Meihui Zhang, Gang Chen, Qian Lin, and Beng Chin Ooi. 2021. Blockchains vs. Distributed Databases: Dichotomy and Fusion. In *Proceedings of the 2021 International Conference on Management of Data*. 1504–1517.
- [66] Pingcheng Ruan, Dumitrel Loghin, Quang-Trung Ta, Meihui Zhang, Gang Chen, and Beng Chin Ooi. 2020. A transactional perspective on execute-order-validate blockchains. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 543–557.
- [67] Signe Rüsçh, Kai Bleeke, and Rüdiger Kapitza. 2019. Bloxy: Providing transparent and generic BFT-based ordering services for blockchains. In *2019 38th Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 305–30509.
- [68] Mark Russinovich, Edward Ashton, Christine Avanesians, Miguel Castro, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Cédric Fournet, Matthew Kerner, Sid Krishna, et al. 2019. CCF: A framework for building confidential verifiable replicated services. *Technical report, Microsoft Research and Microsoft Azure* (2019).
- [69] Felix Martin Schuhknecht, Ankur Sharma, Jens Dittrich, and Divya Agrawal. 2021. chainifyDB: How to get rid of your Blockchain and use your DBMS instead. In *11th Conference on Innovative Data Systems Research, CIDR 2021, Virtual Event, January 11-15, 2021, Online Proceedings*. [www.cidrdb.org](http://cidrdb.org). http://cidrdb.org/cidr2021/papers/cidr2021_paper04.pdf
- [70] Ankur Sharma, Felix Martin Schuhknecht, Divya Agrawal, and Jens Dittrich. 2019. Blurring the lines between blockchains and database systems: the case of hyperledger fabric. In *Proceedings of the 2019 International Conference on Management of Data*. 105–122.
- [71] Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolić. 2019. Mir-bft: High-throughput robust bft for decentralized networks. *arXiv preprint arXiv:1906.05552* (2019).
- [72] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. 2021. Basil: Breaking up BFT with ACID (transactions). In *SOSP '21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, Robbert van Renesse and Nikolai Zeldovich (Eds.). ACM, 1–17. <https://doi.org/10.1145/3477132.3483552>

- [73] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 1–12.
- [74] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 18–32.
- [75] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores. *Proceedings of the VLDB Endowment* 10, 2 (2016), 49–60.
- [76] Zhaoguo Wang, Shuai Mu, Yang Cui, Han Yi, Haibo Chen, and Jinyang Li. 2016. Scaling multicore databases via constrained parallel execution. In *Proceedings of the 2016 International Conference on Management of Data*. 1643–1658.
- [77] Shan-Hung Wu, Tsai-Yu Feng, Meng-Kai Liao, Shao-Kan Pi, and Yu-Shan Lin. 2016. T-part: Partitioning of transactions for forward-pushing in deterministic database systems. In *Proceedings of the 2016 International Conference on Management of Data*. 1553–1565.
- [78] Karl Wüst and Arthur Gervais. 2018. Do you need a blockchain?. In *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. IEEE, 45–54.
- [79] Dylan Yaga, Peter Mell, Nik Roby, and Karen Scarfone. 2019. Blockchain technology overview. *arXiv preprint arXiv:1906.11078* (2019).
- [80] Chang Yao, Divyakant Agrawal, Gang Chen, Qian Lin, Beng Chin Ooi, Weng-Fai Wong, and Meihui Zhang. 2016. Exploiting single-threaded model in multi-core in-memory systems. *IEEE Transactions on Knowledge and Data Engineering* 28, 10 (2016), 2635–2650.
- [81] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 347–356.
- [82] Xiangyao Yu, Andrew Pavlo, Daniel Sanchez, and Srinivas Devadas. 2016. Tictoc: Time traveling optimistic concurrency control. In *Proceedings of the 2016 International Conference on Management of Data*. 1629–1642.
- [83] Yuan Yuan, Kaibo Wang, Rubao Lee, Xiaoning Ding, Jing Xing, Spyros Blanas, and Xiaodong Zhang. 2016. Bcc: Reducing false aborts in optimistic concurrency control with low cost for in-memory databases. *Proceedings of the VLDB Endowment* 9, 6 (2016), 504–515.
- [84] Yunhao Zhang, Srinath T. V. Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. 2020. Byzantine Ordered Consensus without Byzantine Oligarchy. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 633–649. <https://www.usenix.org/conference/osdi20/presentation/zhang-yunhao>

Received July 2022; revised October 2022; accepted November 2022